

Introduction to statistical learning theory

First-order approximation, gradient descent and neural networks

Carlos Cotrini

ETH Zurich

6 October 2026

1

Section 1

About us

About Carlos



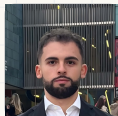
Dr. Carlos Cotrini

- BSc Mathematics, 2009
- MSc Information Security, ETH, 2014
- PhD Information Security, ETH, 2019
- PostDoc in AI and ML, ETH, 2019 to 2021
- Senior scientist, ETH, 2021 to now

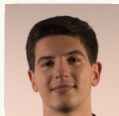
Courses at ETH and UZH

- Advanced machine learning, 2019 to now
- Statistical learning theory, 2020 to 2022
- AI in medicine, 2020 to 2022
- Data analysis in physics, 2020 to 2025
- Stochastics and machine learning, 2024 to now
- Building AI/ML applications, 2024 to now
- AI and IT in industry, 2025 to now
- Software engineering fundamentals, 2025 to now
- AI project, 2026 to now
- Applied introduction to machine learning, from 2027

The teaching team



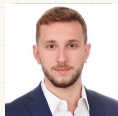
Ivan Angelovski



Amine Benjelloun
Touimi



Ghali Berbich



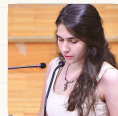
Konstantinos
Chasiotis



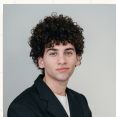
Luca Chimienti



Vassili De Palma



Matina Lampaki



Angelo Marano



Elisej
Orsini-Rosenberg



Frederick Puls



Lamberto Ragnolini



Jesus Solano



Jinlei Zhou

We study how we teach this

Transforming Confusion into Diffusion SIGCSE TS 2026

CER Best Paper (Global) award

- **Full stack ML:** you build the model from the ground up before you reach for a library.
- Trial with $N = 208$: that beat the library first approach by about **10 percent** on conceptual understanding ($p = 0.006$), and left participants more curious.

PATHOS: A Pedagogical Method for Sequencing Instruction in Multi-Foundational Machine Learning ICER 2026

- “...multi-foundational: they draw on multiple distinct fields and have complex prerequisite structures.”
- The order we teach things in is itself designed. Trial with $N = 209$: the designed order scored higher ($p = 0.011$, $d = 0.32$) at lower cognitive load.

Experimental-study team Dr. Sverrir Thorgeirsson · Stefan Cadar · Sofi Mousia · Umut Demirbas

Johannes Schulte-Vels · Nikolaj Murasov · Jonathan Maillefaud · Oscar Jauffret

CSCS AI Tutorials, Part I, Statistical learning theory

Transforming Confusion into Diffusion: Advancing Machine Learning Education via Bottom-Up Instruction

Carlos Cotrill

ETH Zurich

Zurich, Switzerland

cotril@inf.ethz.ch

Jana Selam

ETH Zurich

Zurich, Switzerland

jana.selam@ethz.ch

Sverrir Thorgeirsson

ETH Zurich

Zurich, Switzerland

sverrir.thorgeirsson@ethz.ch

Zhenqiang Su

ETH Zurich

Zurich, Switzerland

zhensu@inf.ethz.ch

Abstract

Relating conceptual depth with practical skill development is a persistent challenge in advanced machine learning (ML) education, where powerful frameworks can obscure underlying mathematical and computational principles. To address this, we crafted a new pedagogical approach that we call full-stack machine learning (FSML), which emphasizes the construction of large language models and diffusion models from scratch. To evaluate the effectiveness of FSML, we conducted a classroom-based randomized controlled trial (RCT) in which FSML-based instruction was compared against a popular library-based instructional approach. We measured the study's conceptual understanding through a specialized assessment and administered a survey capturing knowledge gaps, resources, interests, and cognitive load. We found that students who received FSML instruction performed approximately 10% better than control participants on a quiz on transformers and stable diffusion (p = 0.040). They also showed increased curiosity and more positive affective responses, suggesting deeper engagement with ML fundamentals. Our findings indicate that our full-stack approach is effective in improving individual learning outcomes, previously including curriculum for ML and other advanced computing topics.

ACM Reference Format

Cotrill, Carlos; Selam, Jana; Thorgeirsson, Sverrir; and Su, Zhenqiang. 2026. Transforming Confusion into Diffusion: Advancing Machine Learning Education via Bottom-Up Instruction. In SIGCSE Technical Surveys, Vol. 53, No. 1, Article 1. 12 pages. <https://doi.org/10.1145/3700000>.

1 Introduction

Computer science (CS) education has many dimensions that are relevant to their colleagues from other disciplines, one of which is how to balance high-level conceptual approaches to the subject with foundational skill development. While top-down approaches that involve collection and memorization of facts are the mainstream and dominant means of education [1, 10], they may leave students short of practical proficiency when not paired with consistent practice in hands-on, a position well known, for example, in foundational library education [15, 21] and mathematics [13, 11]. Consequently, many education have a blend of top-down and bottom-up methods to emphasize on the strength of both, although the best way to integrate these is competing education remains a topic of ongoing discussion [19]. In machine learning (ML), this question is particularly

PATHOS: A Pedagogical Method for Sequencing Instruction in Multi-Foundational Machine Learning

Diego Rivera Garcia

ETH Zurich

Zurich, Switzerland

rivera@inf.ethz.ch

Sverrir Thorgeirsson

ETH Zurich

Zurich, Switzerland

sverrir.thorgeirsson@ethz.ch

Dominik Meier

ETH Zurich

Zurich, Switzerland

domini@inf.ethz.ch

Jana Selam

ETH Zurich

Zurich, Switzerland

jana.selam@ethz.ch

Zhenqiang Su

ETH Zurich

Zurich, Switzerland

zhensu@inf.ethz.ch

Abstract

Designing and Center Modern advanced topics in computer science like ML and diffusion models are multi-foundational: they draw on multiple distinct fields and have complex prerequisite structures. This makes instructional sequencing particularly challenging, yet no systematic method exists for designing presentation orders for such topics. (Objectives) Drawing on knowledge space theory and cognitive load theory, we introduce PATHOS (Pedagogical Ordering for Complex Sequencing), an order-oriented sequencing strategy that is designed to minimize extraneous demand and enhance cognitive load for multi-foundational topics. (Methods) We evaluated PATHOS in a pre-registered, randomized controlled trial (N = 209) in a university lecture course comparing a PATHOS-derived sequence for teaching diffusion models against a

three, grounded procedure for determining presentation order that is applicable beyond diffusion models.

CSCS Concepts

• Social and professional topics → Computing education

Keywords

diffusion models, machine learning, AI education, instructional sequencing, knowledge space theory, cognitive load theory

ACM Reference Format

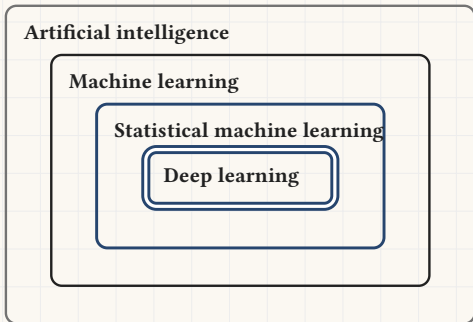
Garcia, Diego; Thorgeirsson, Sverrir; Meier, Dominik; Selam, Jana; and Su, Zhenqiang. 2026. PATHOS: A Pedagogical Method for Sequencing Instruction in Multi-Foundational Machine Learning. In SIGCSE Technical Surveys, Vol. 53, No. 1, Article 1. 12 pages. <https://doi.org/10.1145/3700000>.

2

Section 2

Artificial intelligence, what is it?

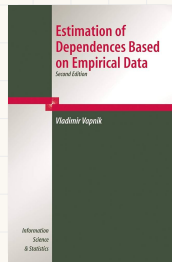
From artificial intelligence to deep learning



- **Artificial intelligence** designs programs that act like intelligent systems, by whichever means.
- **Machine learning** does it by estimating dependencies from observed data.
- **Statistical machine learning** estimates those dependencies with Vapnik's empirical risk minimisation.
- **Deep learning** does the same with deep neural networks.

Vladimir Vapnik, and where this all comes from

- He turned learning from data into a mathematical question.
- His book *Estimation of Dependences Based on Empirical Data* founded statistical machine learning.
- Every model in this course minimises the empirical risk he defined.



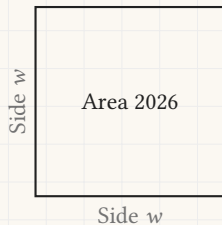
Vladimir N. Vapnik, and the 2006 edition

3

Section 3

First-order approximation

Calculating square roots

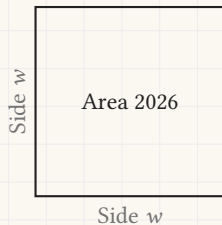


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
-----	-------	------------------	--------

How long is the side?

Calculating square roots

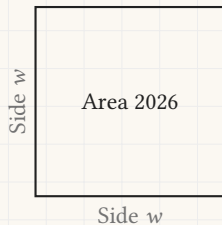


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑

How long is the side?

Calculating square roots

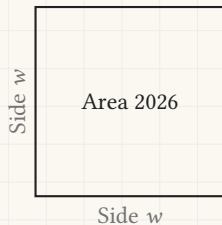


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓

How long is the side?

Calculating square roots

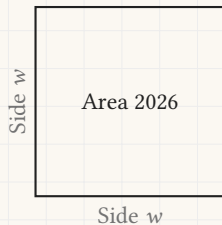


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓
45	2025	+1	↑

How long is the side?

Calculating square roots

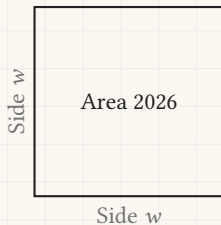


Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓
45	2025	+1	↑
⋮	⋮	⋮	

How long is the side?

Calculating square roots



How long is the side?

Find w with $w^2 = 2026$.

w	w^2	$l = 2026 - w^2$	Update
40	1600	+426	↑
50	2500	-474	↓
45	2025	+1	↑
⋮	⋮	⋮	

Nobody here computed a square root. Each row is a guess that was scored, and the score said which way to move.

The update rule

$$w := w$$

w	w^2	l
-----	-------	-----

w	w^2	l
-----	-------	-----

The update rule

$$w := w + \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
-----	-------	-----

w	w^2	l
-----	-------	-----

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
-----	-------	-----

w	w^2	l
-----	-------	-----

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l

- Starting at $w = 40$, with $\alpha = 0.005$.

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l
45.007	2025.618	0.382
45.009	2025.790	0.210
45.010	2025.884	0.116
45.010	2025.936	0.064
45.011	2025.965	0.035
45.011	2025.981	0.019
45.011	2025.989	0.011
45.011	2025.994	0.006
45.011	2025.997	0.003
45.011	2025.998	0.002
45.011	2025.999	0.001

- Starting at $w = 40$, with $\alpha = 0.005$.
- The whole run, all 23 rounds of it.
- From the fifth row of this block, w has stopped moving and the verdict is still shrinking.

The update rule

$$w := w + \underbrace{\alpha}_{\text{How far to move}} \underbrace{l}_{\text{The verdict}}$$

w	w^2	l
40.000	1600.000	426.000
42.130	1774.937	251.063
43.385	1882.286	143.714
44.104	1945.153	80.847
44.508	1980.973	45.027
44.733	2001.064	24.936
44.858	2012.234	13.766
44.927	2018.414	7.586
44.965	2021.824	4.176
44.986	2023.702	2.298
44.997	2024.736	1.264
45.003	2025.305	0.695

w	w^2	l
45.007	2025.618	0.382
45.009	2025.790	0.210
45.010	2025.884	0.116
45.010	2025.936	0.064
45.011	2025.965	0.035
45.011	2025.981	0.019
45.011	2025.989	0.011
45.011	2025.994	0.006
45.011	2025.997	0.003
45.011	2025.998	0.002
45.011	2025.999	0.001

- Starting at $w = 40$, with $\alpha = 0.005$.
- The whole run, all 23 rounds of it.
- From the fifth row of this block, w has stopped moving and the verdict is still shrinking.
- $\sqrt{2026} = 45.011110$, and no square root was ever taken.

Iterative problem solving

Iterative problem solving

Guess A value of w , however crude.

Guess

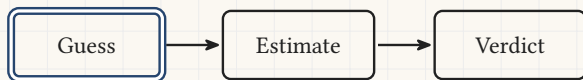
Iterative problem solving



Guess A value of w , however crude.

Estimate What the guess predicts.

Iterative problem solving

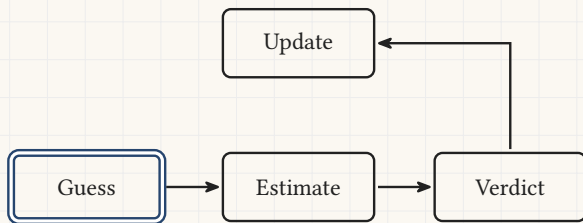


Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

Iterative problem solving



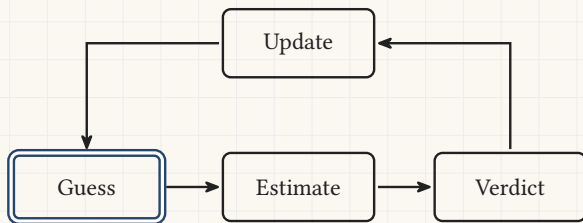
Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

Update Move w the way the verdict says.

Iterative problem solving



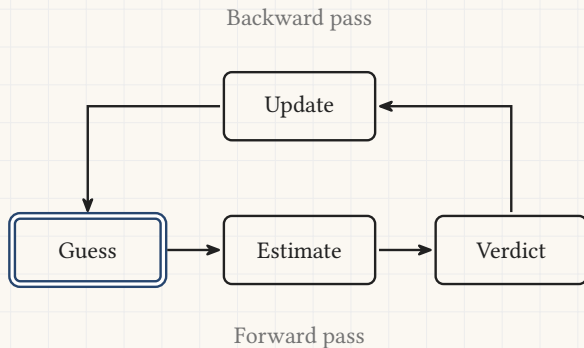
Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

Update Move w the way the verdict says.

Iterative problem solving



Guess A value of w , however crude.

Estimate What the guess predicts.

Verdict The signed miss l .

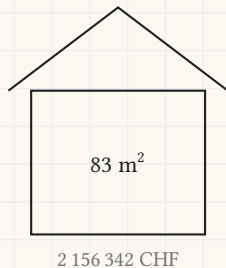
Update Move w the way the verdict says.

4

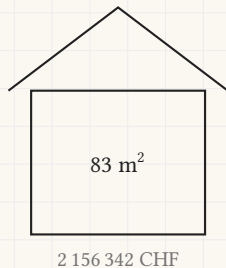
Section 4

Estimating house prices

One house, one measurement

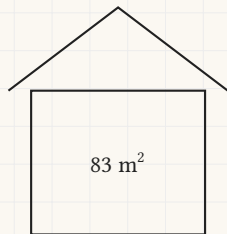


One house, one measurement



Data $x = 83$, the area. $y = 2\,156\,342$, the price.

One house, one measurement

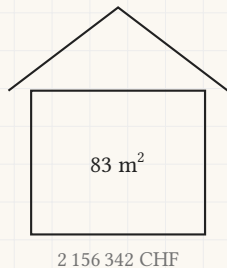


2 156 342 CHF

Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is
 $\hat{y} = w x$.

One house, one measurement

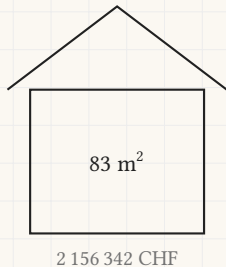


Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is $\hat{y} = w x$.

Verdict The signed miss $l = y - \hat{y}$.

One house, one measurement



Data $x = 83$, the area. $y = 2\,156\,342$, the price.

Hypothesis Price is proportional to area, so the estimate is $\hat{y} = w x$.

Verdict The signed miss $l = y - \hat{y}$.

Problem Find the w that drives l to zero.

One row and one unknown, so the answer is $y/x = 25\,980.02$ francs per square metre, and the loop never divides.

Many houses, one measurement

House	Area	Price
-------	------	-------

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Many houses, one measurement

Hypothesis The same one, row by row: $\hat{y}_i = w x_i$.

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

Hypothesis The same one, row by row: $\hat{y}_i = w x_i$.

Verdict One miss per row, $l_i = y_i - \hat{y}_i$.

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

Many houses, one measurement

House	Area	Price
1	83	2 156 342
2	62	1 461 635
3	104	2 650 166
4	128	3 184 899
5	145	3 651 113

Hypothesis The same one, row by row: $\hat{y}_i = w x_i$.

Verdict One miss per row, $l_i = y_i - \hat{y}_i$.

Problem No single w empties every row. Drive the *average* verdict $\bar{l}$ to zero instead.

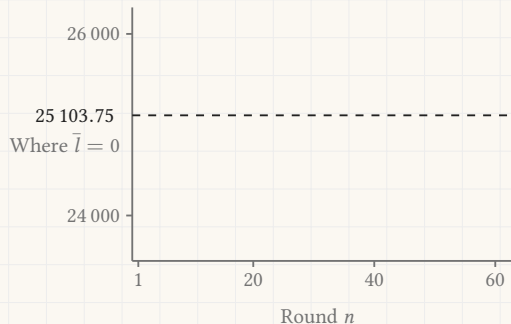
$$\bar{l} = \frac{l_1 + l_2 + \cdots + l_n}{n}$$

House 1 is the house on the last slide. Houses 2 to 5 are synthetic, generated with a fixed seed.

The Robbins-Monro algorithm

One house i at random per round, and its miss alone: $w := w + \alpha_n l_i$.

Robbins and Monro, 1951. With minus a gradient in place of the miss, it is stochastic gradient descent.

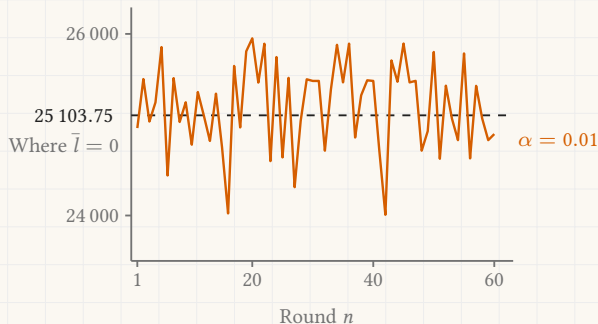


Round	$\alpha = 0.01$	$\alpha_n = 0.01/n$
60		
1 000		
10 000		

The Robbins-Monro algorithm

One house i at random per round, and its miss alone: $w := w + \alpha_n l_i$.

Robbins and Monro, 1951. With minus a gradient in place of the miss, it is stochastic gradient descent.

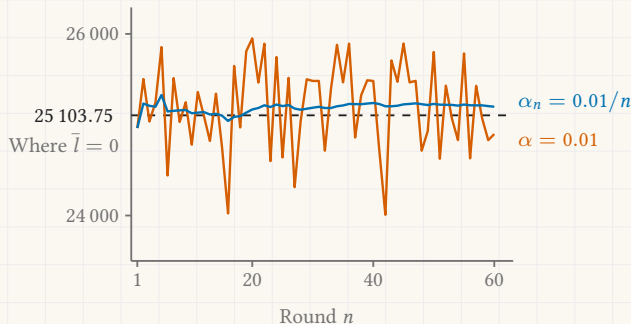


Round	$\alpha = 0.01$	$\alpha_n = 0.01/n$
60	24 896.65	
1 000	25 827.38	
10 000	24 303.55	

The Robbins-Monro algorithm

One house i at random per round, and its miss alone: $w := w + \alpha_n l_i$.

Robbins and Monro, 1951. With minus a gradient in place of the miss, it is stochastic gradient descent.

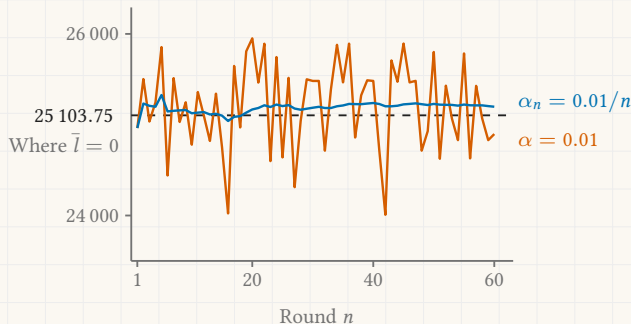


Round	$\alpha = 0.01$	$\alpha_n = 0.01/n$
60	24 896.65	25 198.44
1 000	25 827.38	25 114.23
10 000	24 303.55	25 108.03

The Robbins-Monro algorithm

One house i at random per round, and its miss alone: $w := w + \alpha_n l_i$.

Robbins and Monro, 1951. With minus a gradient in place of the miss, it is stochastic gradient descent.



Round	$\alpha = 0.01$	$\alpha_n = 0.01/n$
60	24 896.65	25 198.44
1 000	25 827.38	25 114.23
10 000	24 303.55	25 108.03

$$\underbrace{\sum_n \alpha_n = \infty}$$

It never
stops short

$$\underbrace{\sum_n \alpha_n^2 < \infty}$$

The summed noise
stays finite

Many measurements: the loop

Many measurements: the loop

Prices

y_1
y_2
$\vdots$
y_n

X

x_{11}	x_{12}	$\cdots$	x_{1d}
x_{21}	x_{22}	$\cdots$	x_{2d}
$\vdots$	$\vdots$	$\ddots$	$\vdots$
x_{n1}	x_{n2}	$\cdots$	x_{nd}

Many measurements: the loop

Prices

y_1
y_2
$\vdots$
y_n

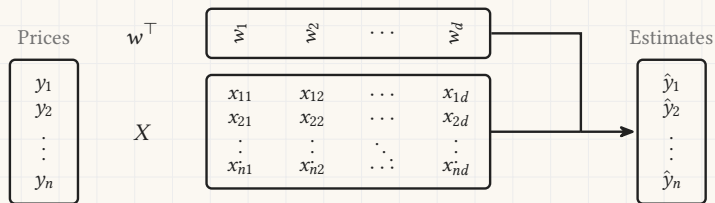
$w^\top$

w_1	w_2	$\dots$	w_d
-------	-------	---------	-------

X

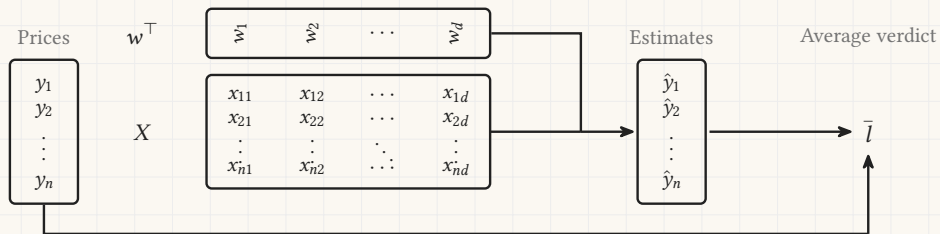
x_{11}	x_{12}	$\dots$	x_{1d}
x_{21}	x_{22}	$\dots$	x_{2d}
$\vdots$	$\vdots$	$\ddots$	$\vdots$
x_{n1}	x_{n2}	$\dots$	x_{nd}

Many measurements: the loop



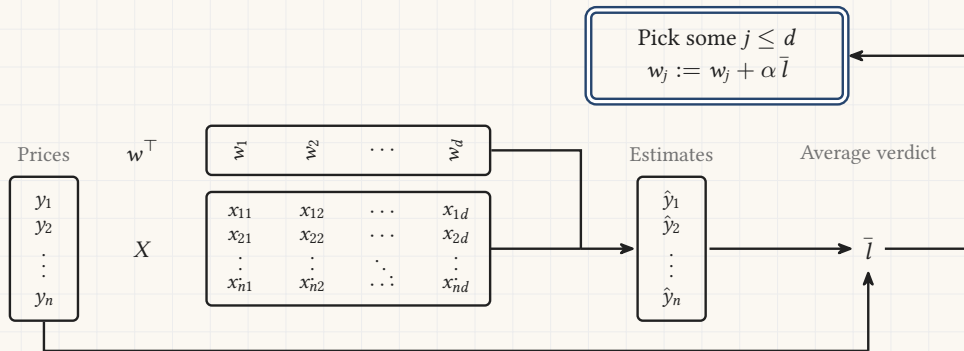
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

Many measurements: the loop



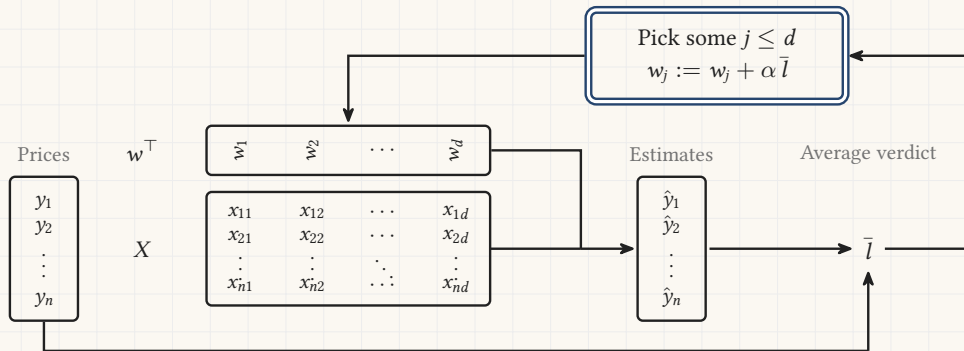
X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

Many measurements: the loop



X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

Many measurements: the loop



X has one row per house and one column per measurement, and $\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_d x_{id}$.

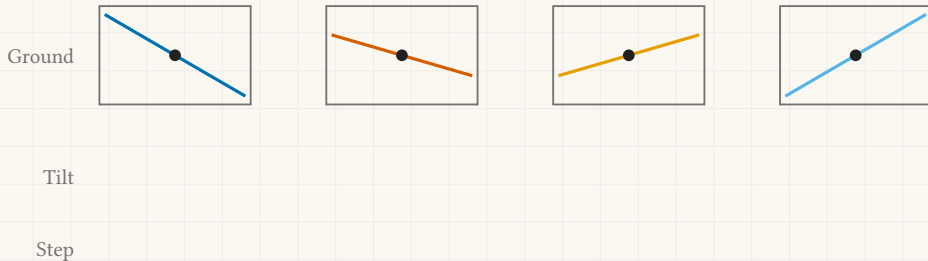
5

Section 5

Gradient descent

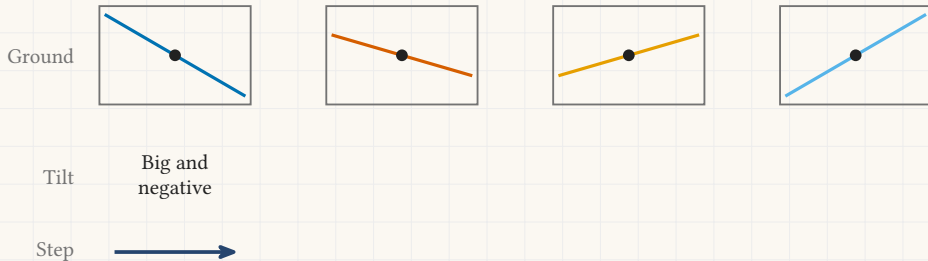
From tilt to step

The ground is the loss $L(w)$, here the average squared miss. One patch of it, tilted four ways.



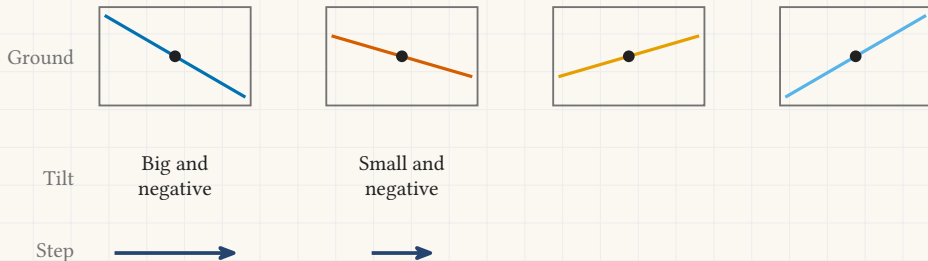
From tilt to step

The ground is the loss $L(w)$, here the average squared miss. One patch of it, tilted four ways.



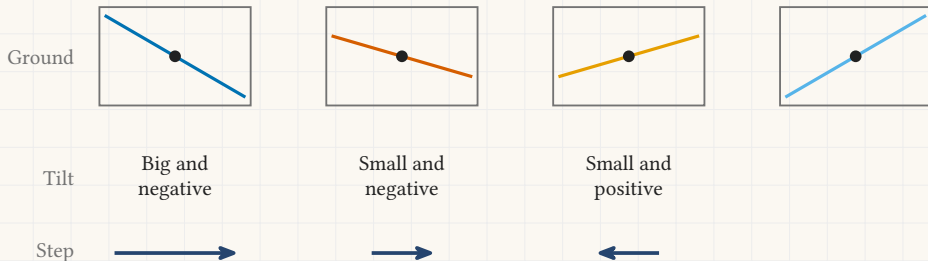
From tilt to step

The ground is the loss $L(w)$, here the average squared miss. One patch of it, tilted four ways.



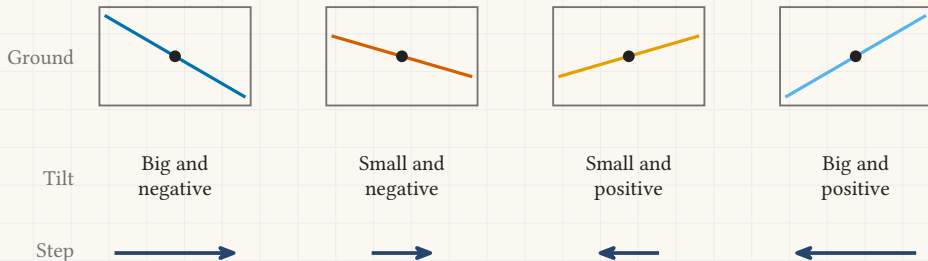
From tilt to step

The ground is the loss $L(w)$, here the average squared miss. One patch of it, tilted four ways.



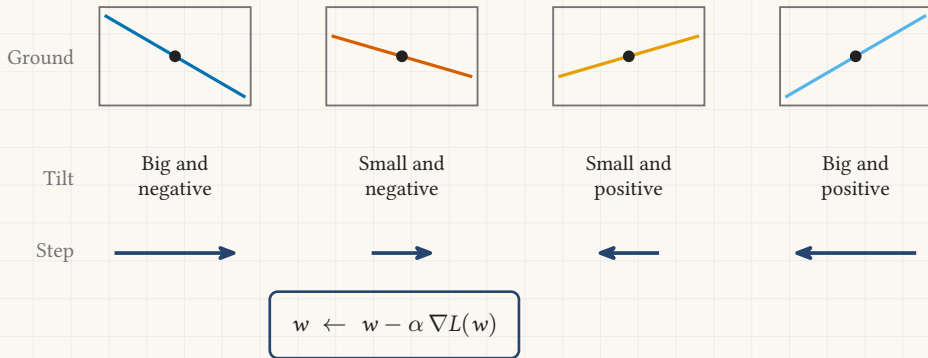
From tilt to step

The ground is the loss $L(w)$, here the average squared miss. One patch of it, tilted four ways.



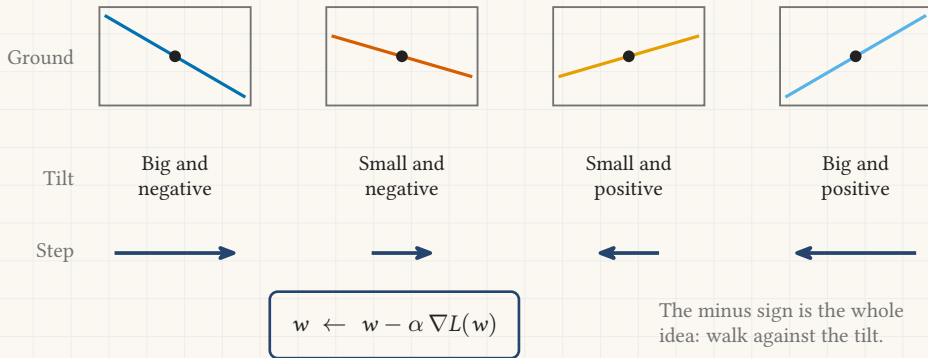
From tilt to step

The ground is the loss $L(w)$, here the average squared miss. One patch of it, tilted four ways.



From tilt to step

The ground is the loss $L(w)$, here the average squared miss. One patch of it, tilted four ways.



Gradient descent in d dimensions

The same rule. The guess is a list, the gradient is a list, and they subtract term by term.

$$w \leftarrow w - \alpha \nabla L(w), \quad w \in \mathbb{R}^d$$

Gradient descent in d dimensions

The same rule. The guess is a list, the gradient is a list, and they subtract term by term.

$$w \leftarrow w - \alpha \nabla L(w), \quad w \in \mathbb{R}^d$$

Coordinate by coordinate, that is d separate copies of the one dimensional rule:

Gradient descent in d dimensions

The same rule. The guess is a list, the gradient is a list, and they subtract term by term.

$$w \leftarrow w - \alpha \nabla L(w), \quad w \in \mathbb{R}^d$$

Coordinate by coordinate, that is d separate copies of the one dimensional rule:

```
for j in range(d):  
    a = L(w_shift(w, j, -eps/2))    # move knob j back by eps/2  
    b = L(w_shift(w, j, +eps/2))    # move knob j on by eps/2  
    grad[j] = (b - a) / eps  
w = [w[j] - alpha * grad[j] for j in range(d)]
```

Gradient descent in d dimensions

The same rule. The guess is a list, the gradient is a list, and they subtract term by term.

$$w \leftarrow w - \alpha \nabla L(w), \quad w \in \mathbb{R}^d$$

Coordinate by coordinate, that is d separate copies of the one dimensional rule:

```
for j in range(d):  
    a = L(w_shift(w, j, -eps/2))    # move knob j back by eps/2  
    b = L(w_shift(w, j, +eps/2))    # move knob j on by eps/2  
    grad[j] = (b - a) / eps  
w = [w[j] - alpha * grad[j] for j in range(d)]
```

That costs $2d$ evaluations of L per step, and for a neural network d runs to millions.

Gradient descent in d dimensions

The same rule. The guess is a list, the gradient is a list, and they subtract term by term.

$$w \leftarrow w - \alpha \nabla L(w), \quad w \in \mathbb{R}^d$$

Coordinate by coordinate, that is d separate copies of the one dimensional rule:

```
for j in range(d):  
    a = L(w_shift(w, j, -eps/2))    # move knob j back by eps/2  
    b = L(w_shift(w, j, +eps/2))    # move knob j on by eps/2  
    grad[j] = (b - a) / eps  
w = [w[j] - alpha * grad[j] for j in range(d)]
```

That costs $2d$ evaluations of L per step, and for a neural network d runs to millions.

Backpropagation gets all d of them for the cost of a few evaluations of L , and PyTorch runs it for you.

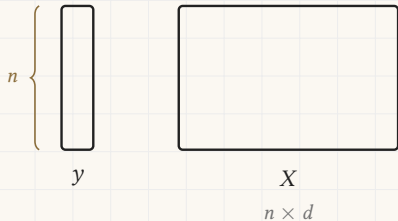
6

Section 6

Neural networks

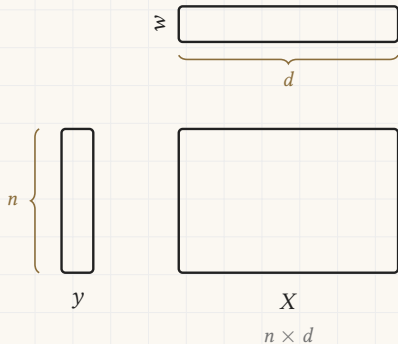
Linear regression with gradient descent

Everything so far, drawn once. Each box is a piece of data, and its shape is the shape of the data.



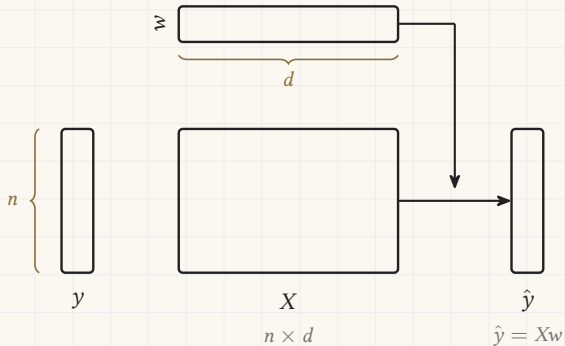
Linear regression with gradient descent

Everything so far, drawn once. Each box is a piece of data, and its shape is the shape of the data.



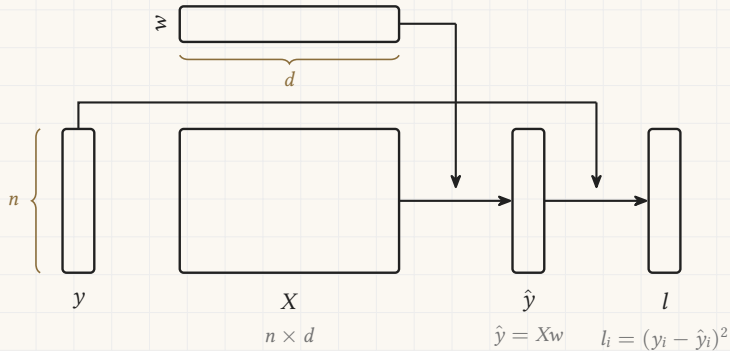
Linear regression with gradient descent

Everything so far, drawn once. Each box is a piece of data, and its shape is the shape of the data.



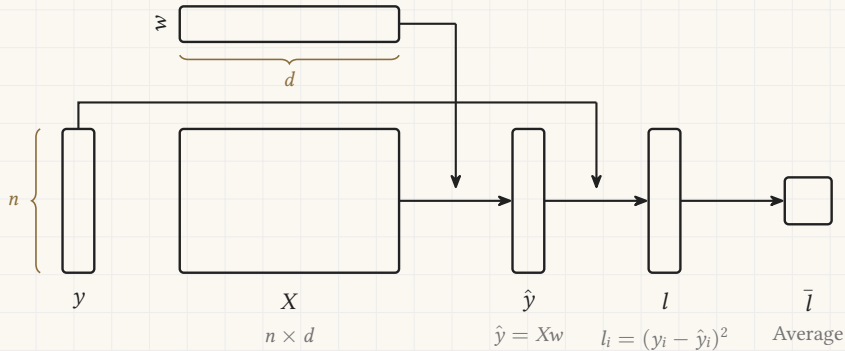
Linear regression with gradient descent

Everything so far, drawn once. Each box is a piece of data, and its shape is the shape of the data.



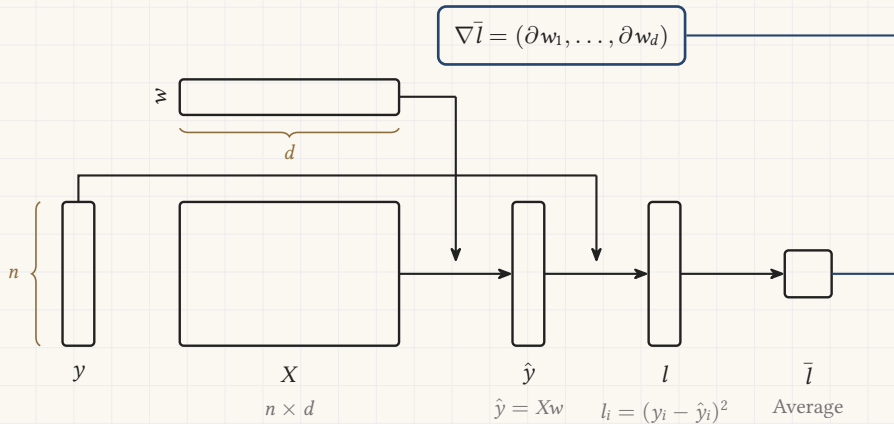
Linear regression with gradient descent

Everything so far, drawn once. Each box is a piece of data, and its shape is the shape of the data.



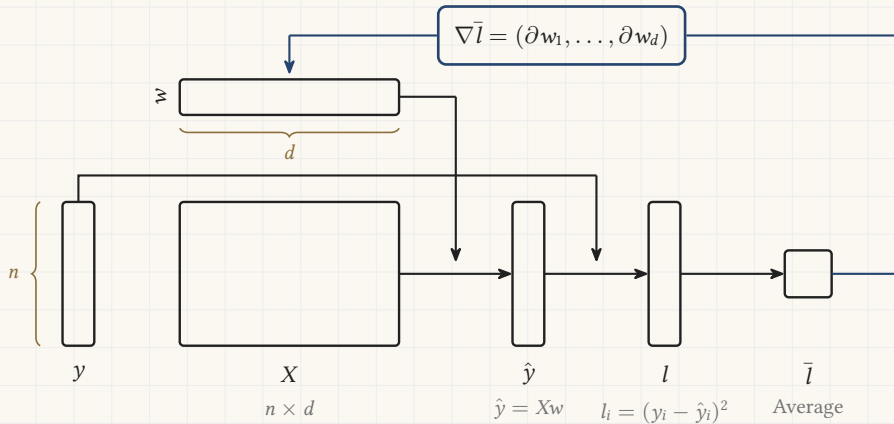
Linear regression with gradient descent

Everything so far, drawn once. Each box is a piece of data, and its shape is the shape of the data.

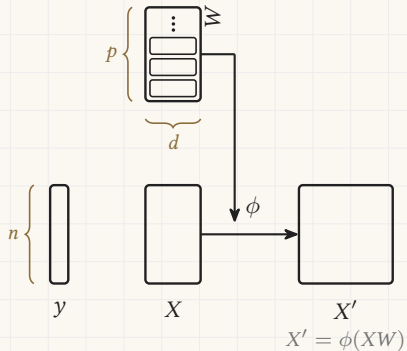


Linear regression with gradient descent

Everything so far, drawn once. Each box is a piece of data, and its shape is the shape of the data.

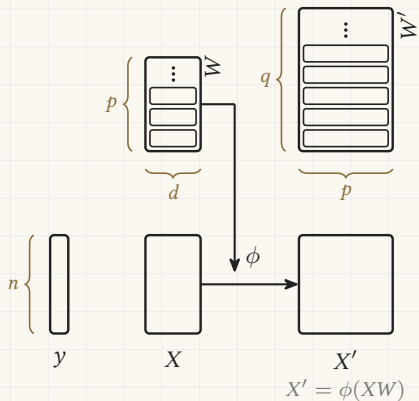


A neural network



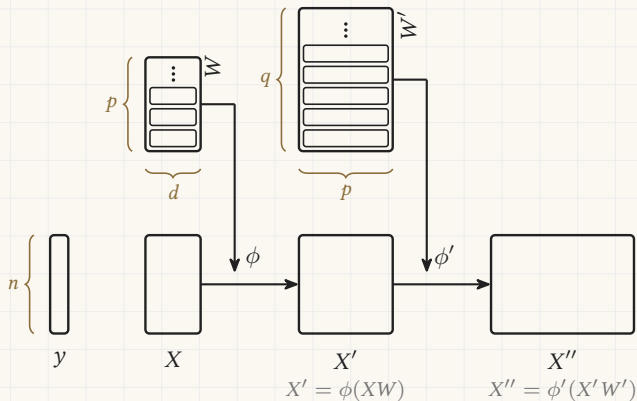
ϕ, ϕ', ϕ'' : activation functions, usually ReLU, $\max(a, 0)$. Here the primes number the layers.

A neural network



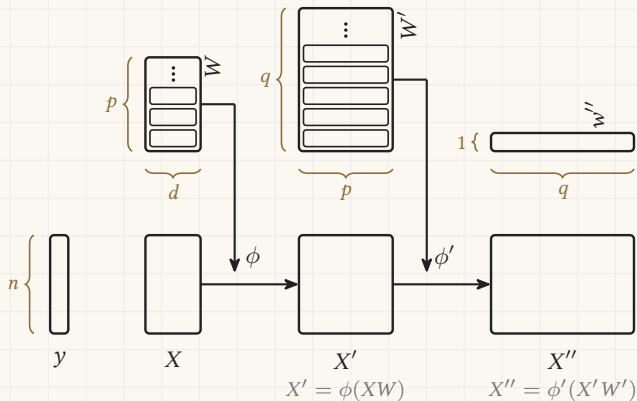
ϕ, ϕ', ϕ'' : activation functions, usually ReLU, $\max(a, 0)$. Here the primes number the layers.

A neural network



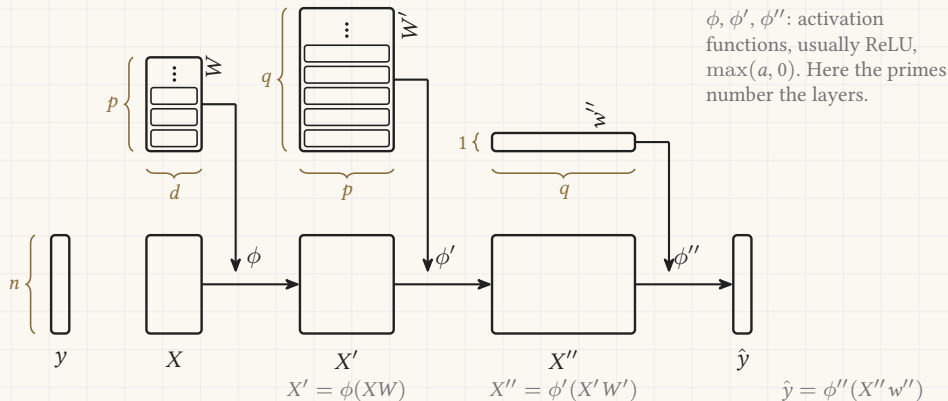
ϕ, ϕ', ϕ'' : activation functions, usually ReLU, $\max(a, 0)$. Here the primes number the layers.

A neural network

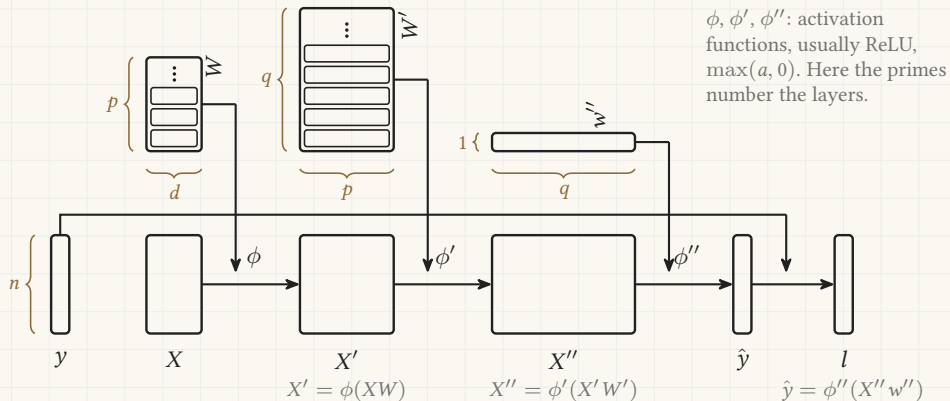


ϕ, ϕ', ϕ'' : activation functions, usually ReLU, $\max(a, 0)$. Here the primes number the layers.

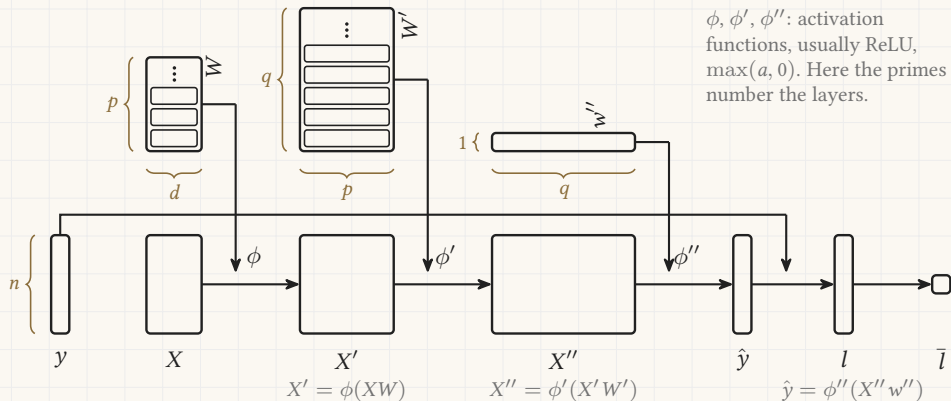
A neural network



A neural network

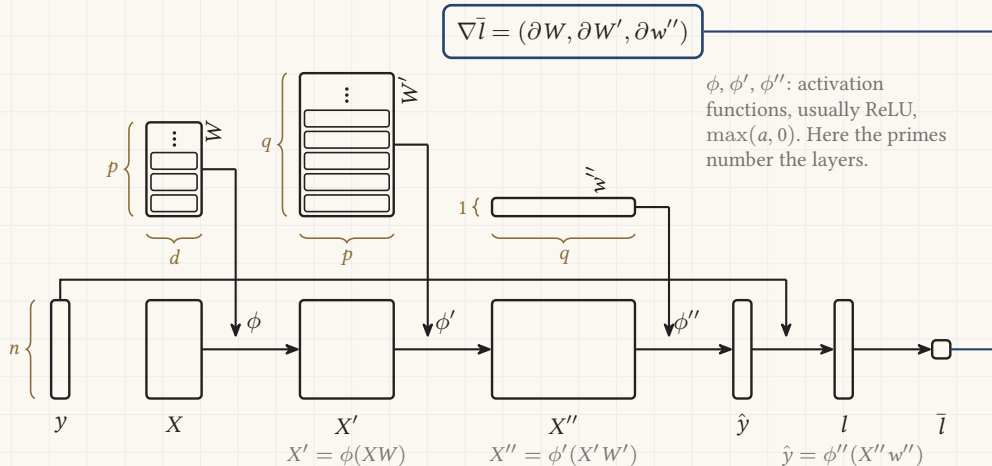


A neural network

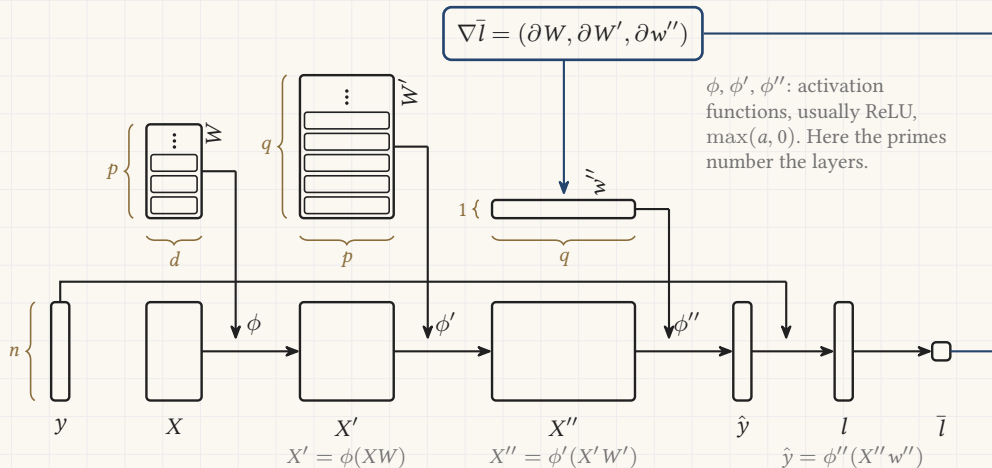


ϕ, ϕ', ϕ'' : activation functions, usually ReLU, $\max(a, 0)$. Here the primes number the layers.

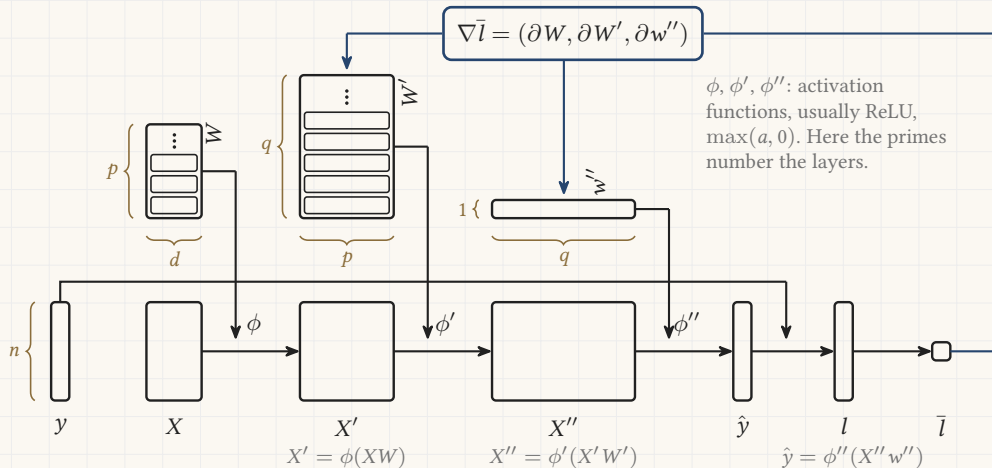
A neural network



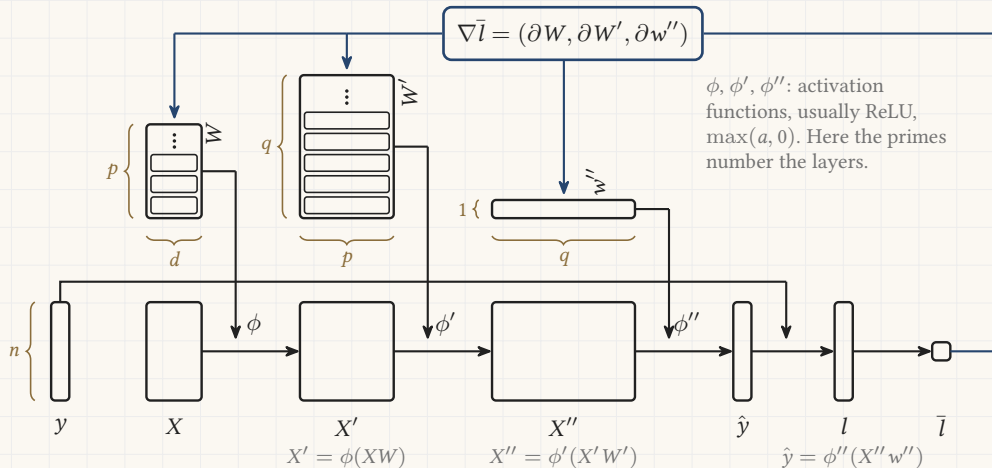
A neural network



A neural network

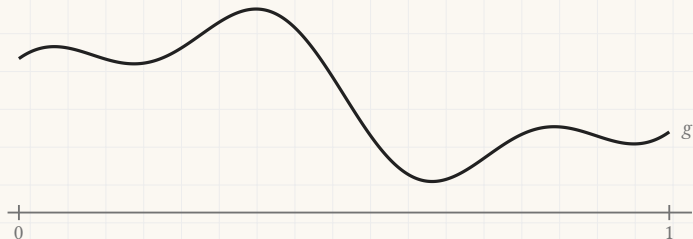


A neural network



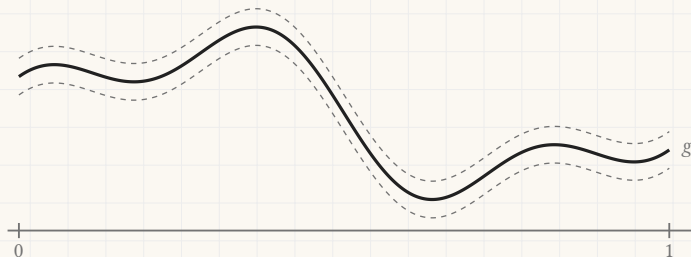
The universal approximation theorem

A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



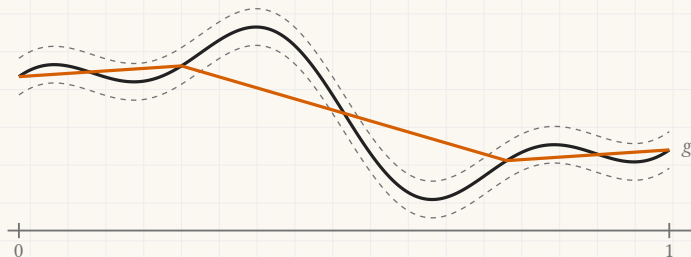
The universal approximation theorem

A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



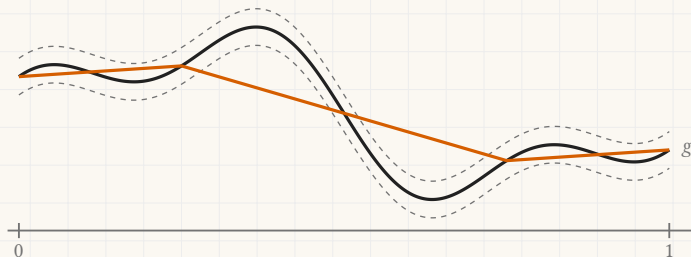
The universal approximation theorem

A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



The universal approximation theorem

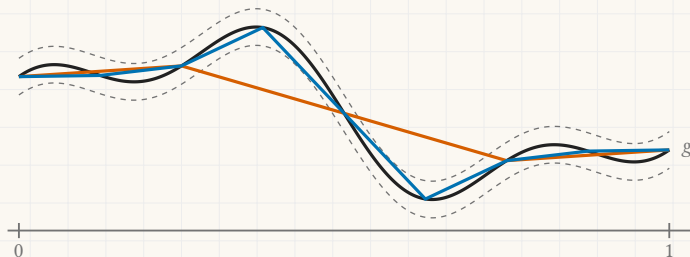
A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



$p = 4$, largest miss 0.85043

The universal approximation theorem

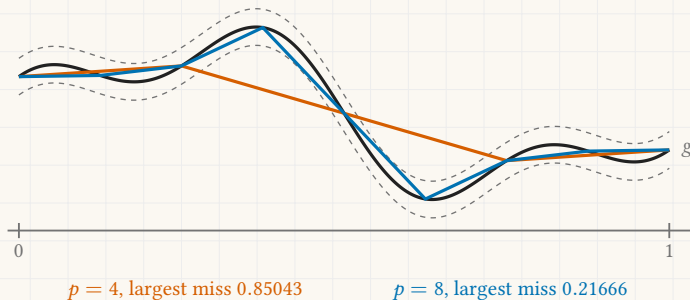
A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



$p = 4$, largest miss 0.85043

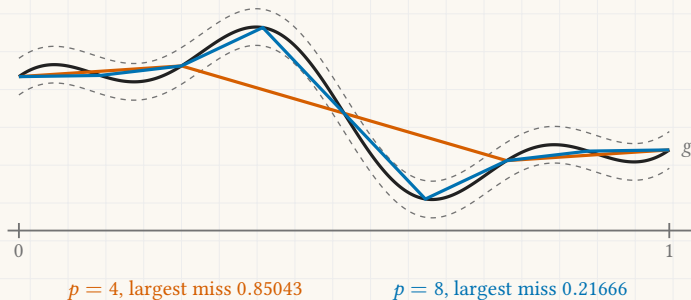
The universal approximation theorem

A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



The universal approximation theorem

A continuous target, and networks with p hidden units drawn over it. The band is the accuracy $\varepsilon = 0.25$.



A network with one hidden layer, and enough units in it, is a universal approximator.

The largest miss falls to 0.06944 at $p = 16$, to 0.01788 at $p = 32$ and to 0.00448 at $p = 64$.

Cybenko, 1989. Hornik, Stinchcombe and White, 1989.

7

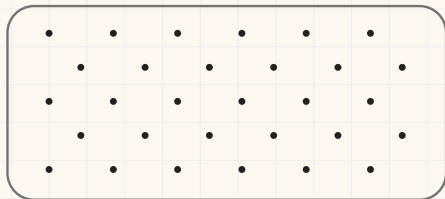
Section 7

The same loop, elsewhere

The number we want and the number we have

$\mathcal{M}$ is a **candidate model**: any rule that turns a row of data into an estimate.

Every case the world will hand us



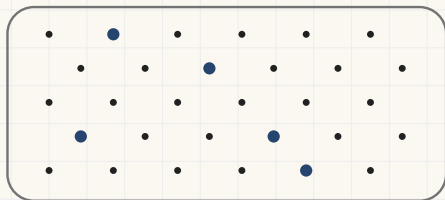
$R(\mathcal{M})$, the average miss over all of them

Nobody can compute this number

The number we want and the number we have

$\mathcal{M}$ is a **candidate model**: any rule that turns a row of data into an estimate.

Every case the world will hand us



$R(\mathcal{M})$, the average miss over all of them

Nobody can compute this number

The n rows we collected

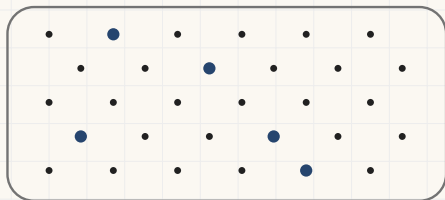


$\bar{l}(\mathcal{M})$, the average miss over those

The number we want and the number we have

$\mathcal{M}$ is a **candidate model**: any rule that turns a row of data into an estimate.

Every case the world will hand us



$R(\mathcal{M})$, the average miss over all of them

Nobody can compute this number

The n rows we collected



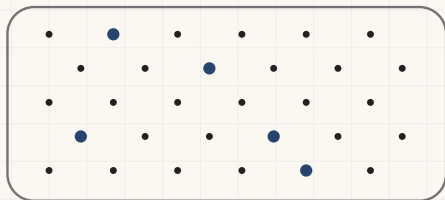
$\bar{l}(\mathcal{M})$, the average miss over those

Vapnik's name for it: the empirical risk

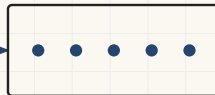
The number we want and the number we have

$\mathcal{M}$ is a **candidate model**: any rule that turns a row of data into an estimate.

Every case the world will hand us



The n rows we collected



$R(\mathcal{M})$, the average miss over all of them

Nobody can compute this number

$\bar{l}(\mathcal{M})$, the average miss over those

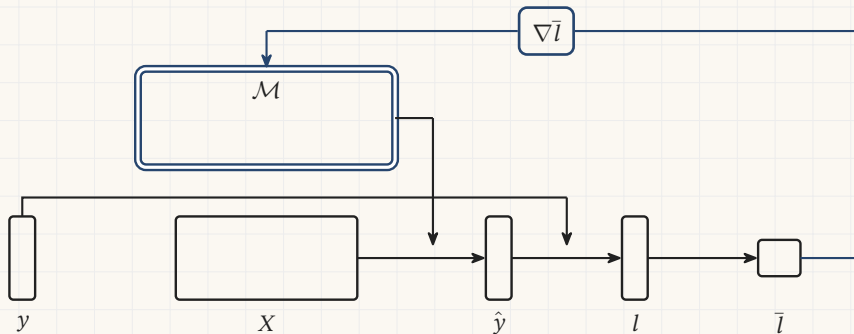
Vapnik's name for it: the empirical risk

Empirical risk minimisation: choose the $\mathcal{M}$ that makes $\bar{l}$ small, and take that as the answer to a question about R .

The loop, with the model as one box

The same picture, with $\mathcal{M}$ where the network was.

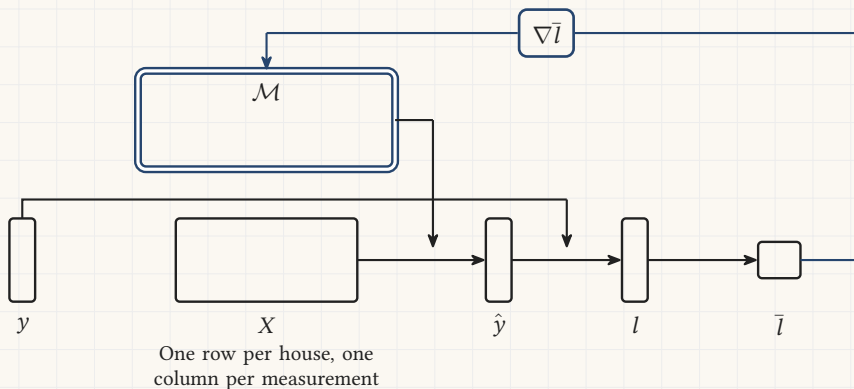
The five houses of this hour, and the network of the last section.



The loop, with the model as one box

The same picture, with $\mathcal{M}$ where the network was.

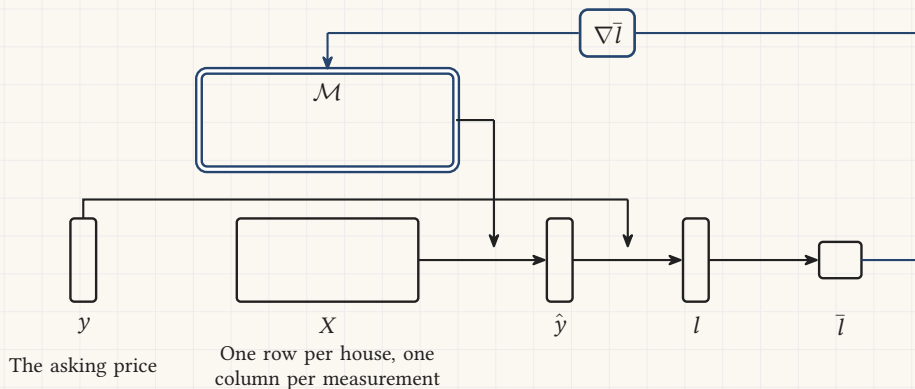
The five houses of this hour, and the network of the last section.



The loop, with the model as one box

The same picture, with $\mathcal{M}$ where the network was.

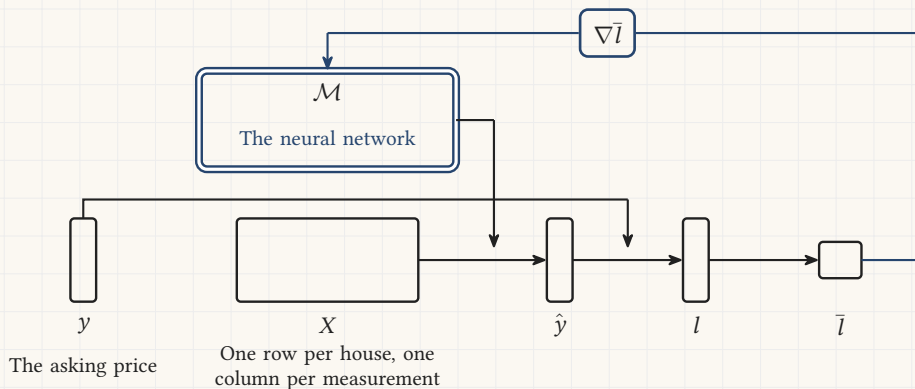
The five houses of this hour, and the network of the last section.



The loop, with the model as one box

The same picture, with $\mathcal{M}$ where the network was.

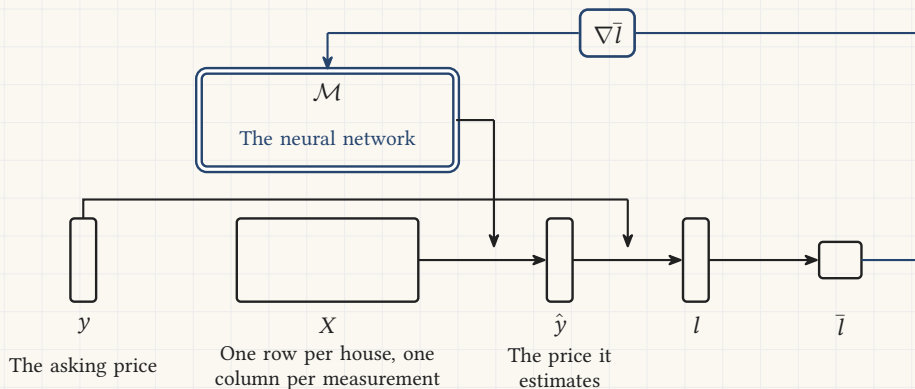
The five houses of this hour, and the network of the last section.



The loop, with the model as one box

The same picture, with $\mathcal{M}$ where the network was.

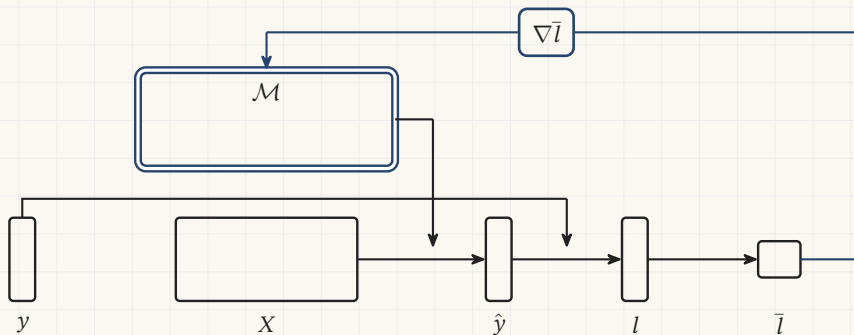
The five houses of this hour, and the network of the last section.



Large language models

A row is text with its next token cut off. Every position of every document gives one.

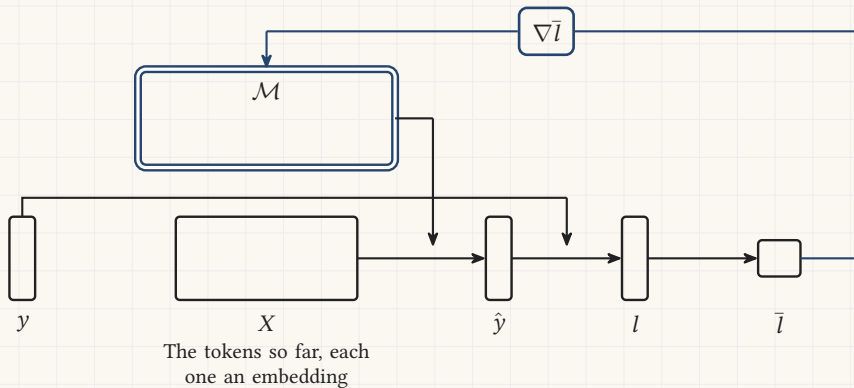
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



Large language models

A row is text with its next token cut off. Every position of every document gives one.

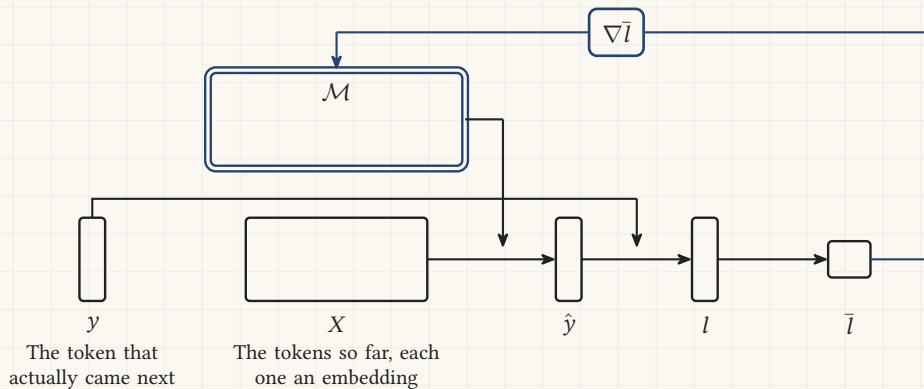
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



Large language models

A row is text with its next token cut off. Every position of every document gives one.

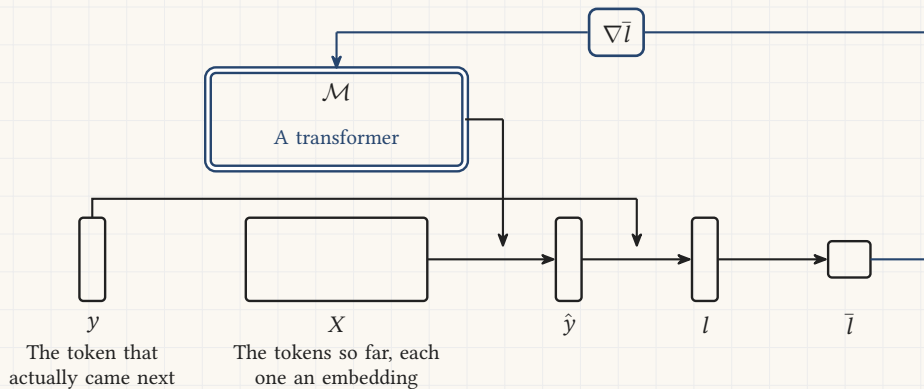
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



Large language models

A row is text with its next token cut off. Every position of every document gives one.

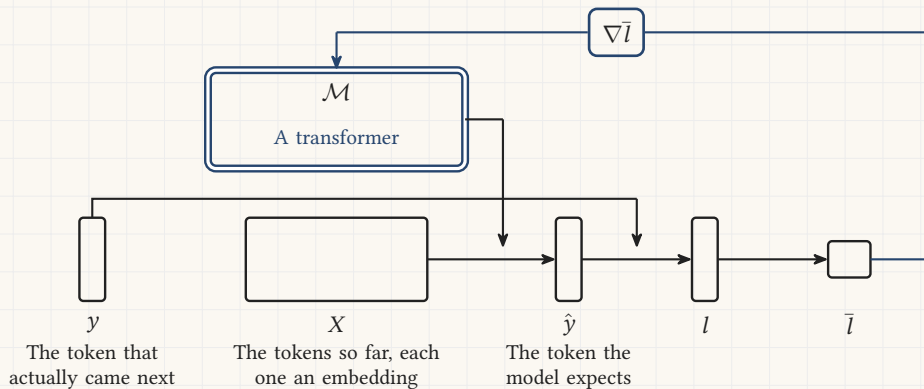
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



Large language models

A row is text with its next token cut off. Every position of every document gives one.

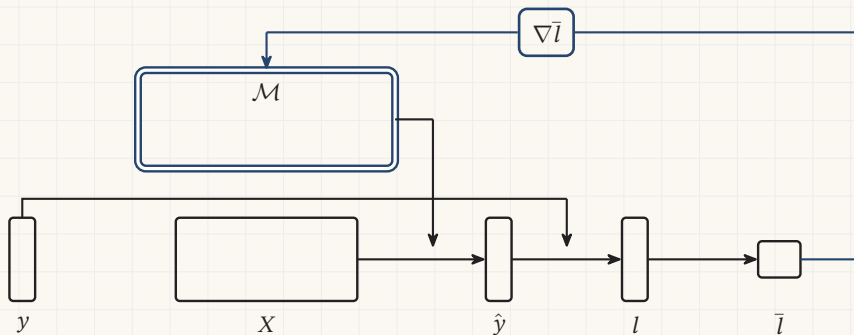
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



Instruction tuning

GPT-3 was finished in 2020 and it completed text. A smaller second pass made it answer.

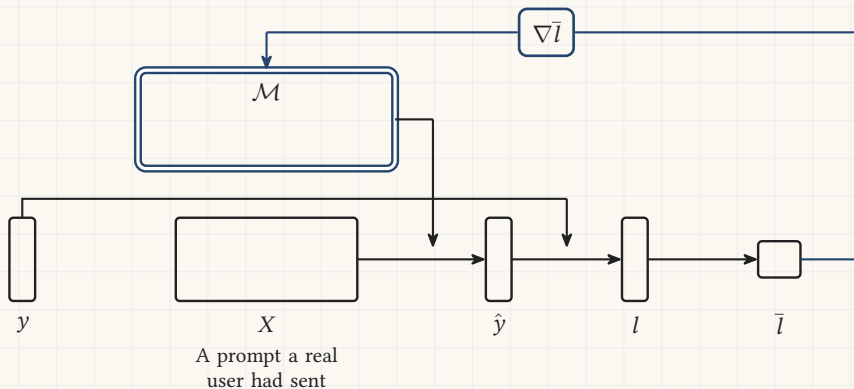
12 725 training prompts, and its replies were preferred to a model 100 times larger. Ouyang and others, 2022.



Instruction tuning

GPT-3 was finished in 2020 and it completed text. A smaller second pass made it answer.

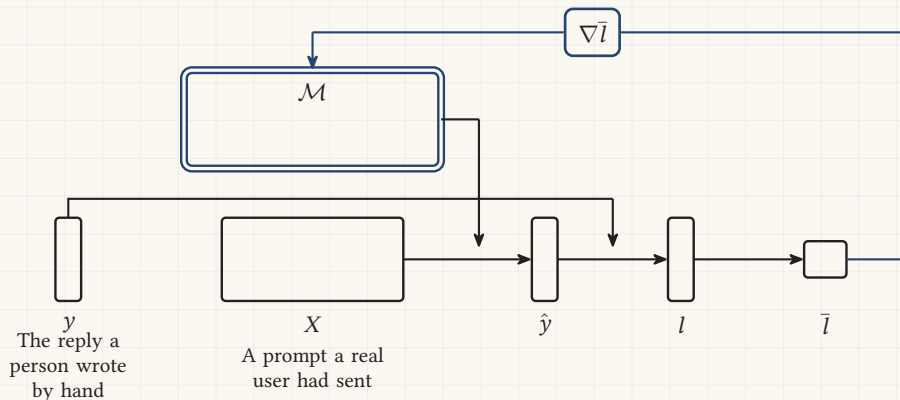
12 725 training prompts, and its replies were preferred to a model 100 times larger. Ouyang and others, 2022.



Instruction tuning

GPT-3 was finished in 2020 and it completed text. A smaller second pass made it answer.

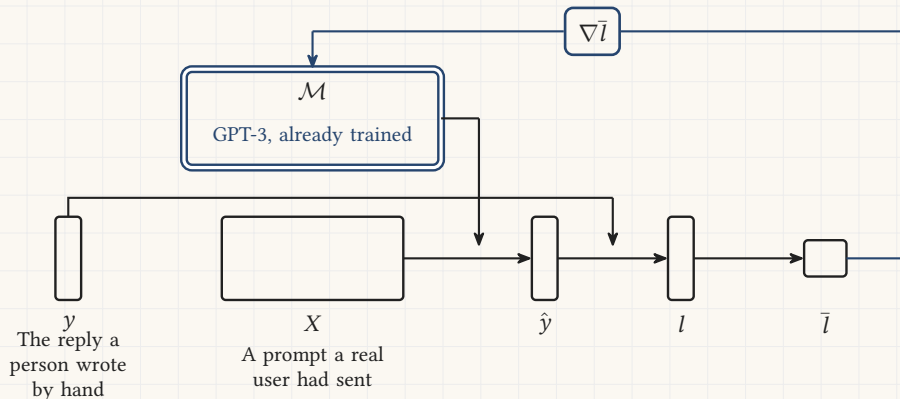
12 725 training prompts, and its replies were preferred to a model 100 times larger. Ouyang and others, 2022.



Instruction tuning

GPT-3 was finished in 2020 and it completed text. A smaller second pass made it answer.

12 725 training prompts, and its replies were preferred to a model 100 times larger. Ouyang and others, 2022.



Instruction tuning

GPT-3 was finished in 2020 and it completed text. A smaller second pass made it answer.

12 725 training prompts, and its replies were preferred to a model 100 times larger. Ouyang and others, 2022.

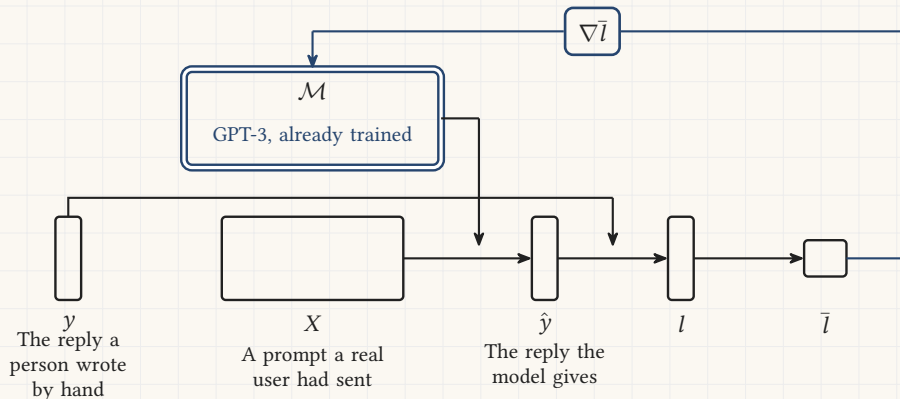


Image generation

Add a measured amount of noise to a picture, then ask which noise was added.

The loss here really is the squared miss. Ho, Jain and Abbeel, 2020; Rombach and others, 2022.

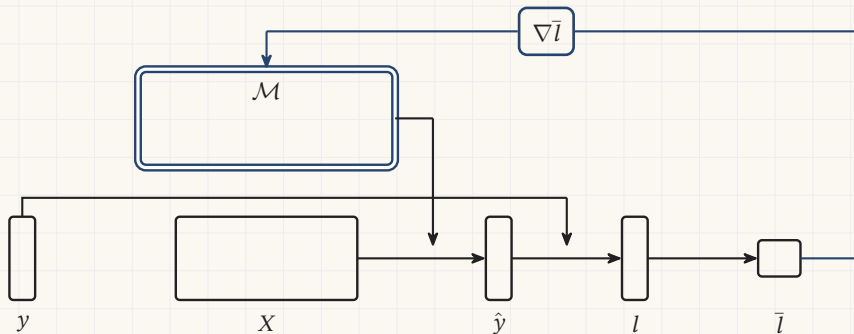


Image generation

Add a measured amount of noise to a picture, then ask which noise was added.

The loss here really is the squared miss. Ho, Jain and Abbeel, 2020; Rombach and others, 2022.

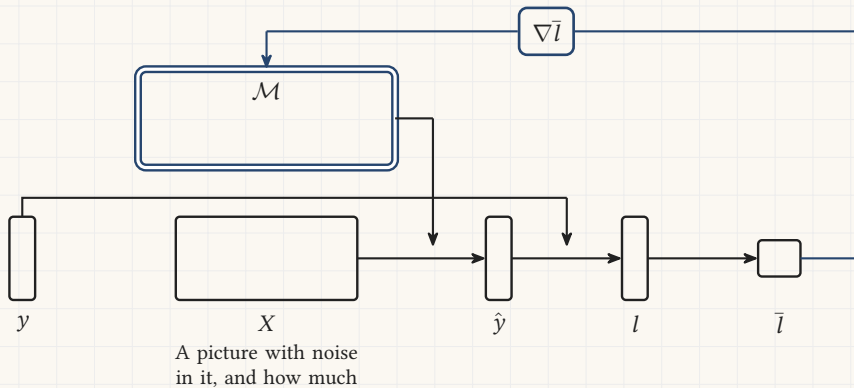


Image generation

Add a measured amount of noise to a picture, then ask which noise was added.

The loss here really is the squared miss. Ho, Jain and Abbeel, 2020; Rombach and others, 2022.

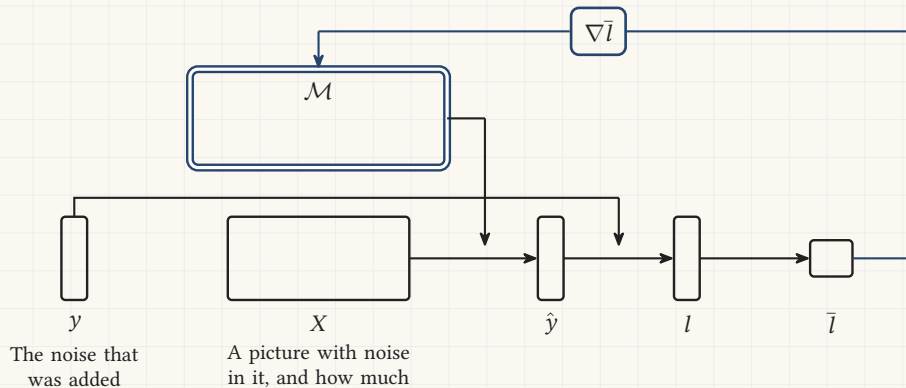


Image generation

Add a measured amount of noise to a picture, then ask which noise was added.

The loss here really is the squared miss. Ho, Jain and Abbeel, 2020; Rombach and others, 2022.

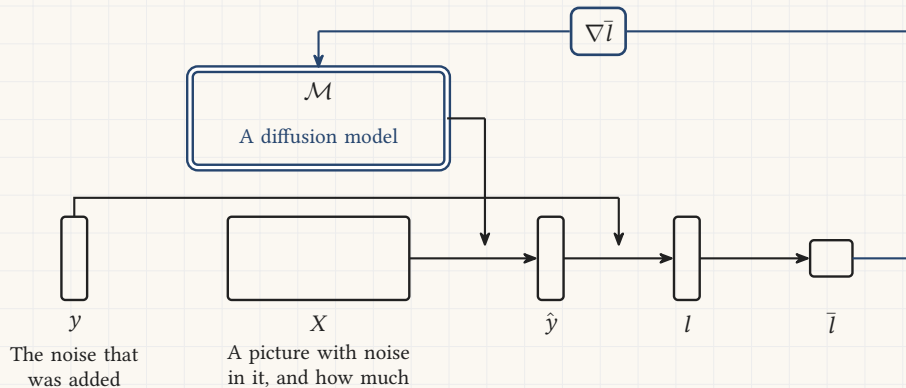
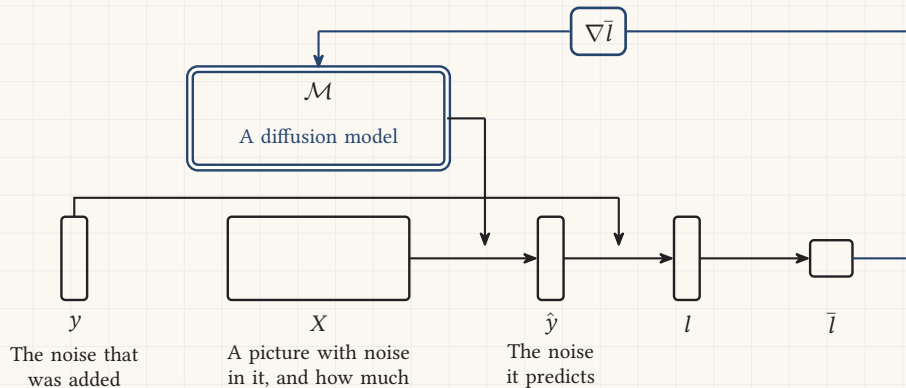


Image generation

Add a measured amount of noise to a picture, then ask which noise was added.

The loss here really is the squared miss. Ho, Jain and Abbeel, 2020; Rombach and others, 2022.



What replaces what

The data, as a block of numbers

`torch.tensor(...)`

What replaces what

The data, as a block of numbers

`torch.tensor(...)`

The network of boxes

`nn.Linear`, `nn.ReLU`, `nn.Module`

What replaces what

The data, as a block of numbers

`torch.tensor(...)`

The network of boxes

`nn.Linear`, `nn.ReLU`, `nn.Module`

The verdict, the average squared miss

`nn.MSELoss()`

What replaces what

The data, as a block of numbers

`torch.tensor(...)`

The network of boxes

`nn.Linear, nn.ReLU, nn.Module`

The verdict, the average squared miss

`nn.MSELoss()`

The gradient

`loss.backward()`

What replaces what

The data, as a block of numbers

`torch.tensor(...)`

The network of boxes

`nn.Linear, nn.ReLU, nn.Module`

The verdict, the average squared miss

`nn.MSELoss()`

The gradient

`loss.backward()`

$w = w - \alpha * g$

`optimizer.step()`

What replaces what

The data, as a block of numbers

`torch.tensor(...)`

The network of boxes

`nn.Linear, nn.ReLU, nn.Module`

The verdict, the average squared miss

`nn.MSELoss()`

The gradient

`loss.backward()`

$w = w - \alpha * g$

`optimizer.step()`

The step size, α

`optim.SGD(..., lr=alpha)`

What replaces what

The data, as a block of numbers

`torch.tensor(...)`

The network of boxes

`nn.Linear, nn.ReLU, nn.Module`

The verdict, the average squared miss

`nn.MSELoss()`

The gradient

`loss.backward()`

$w = w - \alpha * g$

`optimizer.step()`

The step size, α

`optim.SGD(..., lr=alpha)`

One random house per round

`DataLoader(..., shuffle=True)`

What replaces what

The data, as a block of numbers

`torch.tensor(...)`

The network of boxes

`nn.Linear, nn.ReLU, nn.Module`

The verdict, the average squared miss

`nn.MSELoss()`

The gradient

`loss.backward()`

$w = w - \alpha * g$

`optimizer.step()`

The step size, α

`optim.SGD(..., lr=alpha)`

One random house per round

`DataLoader(..., shuffle=True)`

The shrinking step, α_n

`optim.lr_scheduler`

References

- Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics* 22(3), 400 to 407, 1951. doi.org/10.1214/aoms/1177729586
- Vladimir N. Vapnik. *Estimation of Dependences Based on Empirical Data*. Springer, 1982; second edition 2006. doi.org/10.1007/0-387-34239-7
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* 2(4), 303 to 314, 1989. doi.org/10.1007/BF02551274
- Kurt Hornik, Maxwell Stinchcombe and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks* 2(5), 359 to 366, 1989. [doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- Ashish Vaswani and others. Attention is all you need. *Advances in Neural Information Processing Systems* 30, 2017. arxiv.org/abs/1706.03762
- Long Ouyang and others. Training language models to follow instructions with human feedback, 2022. arxiv.org/abs/2203.02155
- Jonathan Ho, Ajay Jain and Pieter Abbeel. Denoising diffusion probabilistic models. *NeurIPS* 33, 2020. arxiv.org/abs/2006.11239
- Robin Rombach and others. High-resolution image synthesis with latent diffusion models. *CVPR*, 2022. arxiv.org/abs/2112.10752